Internationalization in Java

Junior Level: Unicode, Locale Data and Time in Depth

Haluk O. Bingol

Faculty of Computer and Information Sciences
Yeditepe University

v2026-10-15

Outline

1. Unicode in Depth
2. Locale Data in Depth
3. Plural Rules
4. Bundle Lookup in Detail
5. Time Zones in Depth
6. Formatters and Threads
7. Text Direction
8. Summary

Unicode in Depth

Four ways to count a string

unit	Java
byte	`getBytes(UTF_8).length`
`char` (UTF-16 unit)	`length()`, `charAt(i)`
code point	`codePointCount`, `codePoints()`
grapheme	`BreakIterator.getCharacterInstance()`

A **grapheme** is what a reader sees as one character: é written as e + accent, or a thumbs-up with a skin tone (2 code points, 4 `chars`).

- Code points above U+FFFF (most emoji, historic scripts, rare CJK) need **two** `chars`: a *surrogate pair*.
- `substring`, `charAt`, `reverse` can cut a character in half.
- Cursor movement, "max 20 characters", truncation with … should count **graphemes**.
- Since Java 20, `BreakIterator` follows the Unicode rules for emoji sequences.

Counting in practice

codeoutput/Units.java — Java

```
 9
10      static int graphemes(String s) {                    // what a reader calls "characters"
11          BreakIterator it = BreakIterator.getCharacterInstance();
12          it.setText(s);
13          int n = 0;
14          while (it.next() != BreakIterator.DONE) n++;
15          return n;
16      }
17      public static void main(String[] args) {
18          String[] samples = {"ağaç", "Cafe\u0301", "a\uD83D\uDC4D\uD83C\uDFFDb"};  // last: thumbs up + skin tone
19          for (String s : samples)
20              System.out.printf("%-22s chars=%d code points=%d graphemes=%d%n", hex(s),
21                  s.length(), s.codePointCount(0, s.length()), graphemes(s));
22          String t = samples[2];
23          System.out.println("charAt(1) = " + Integer.toHexString(t.charAt(1))
24              + " (half of a surrogate pair)");
25      }
26  }
```

Thumbs-up + skin tone: 6 chars, 4 code points, 3 graphemes.

Counting in practice

codeoutput/Units.java — Java

```
 9
10      static int graphemes(String s) {                    // what a reader calls "characters"
11          BreakIterator it = BreakIterator.getCharacterInstance();
12          it.setText(s);
13          int n = 0;
14          while (it.next() != BreakIterator.DONE) n++;
15          return n;
16      }
17      public static void main(String[] args) {
18          String[] samples = {"ağaç", "Cafe\u0301", "a\uD83D\uDC4D\uD83C\uDFFDb"};  // last: thumbs up + skin tone
19          for (String s : samples)
20              System.out.printf("%-22s chars=%d code points=%d graphemes=%d%n", hex(s),
21                  s.length(), s.codePointCount(0, s.length()), graphemes(s));
22          String t = samples[2];
23          System.out.println("charAt(1) = " + Integer.toHexString(t.charAt(1))
24              + " (half of a surrogate pair)");
25      }
26  }
```

Terminal — codeoutput/Units.txt

```
61 11F 61 E7           chars=4 code points=4 graphemes=4
43 61 66 65 301        chars=5 code points=5 graphemes=4
61 1F44D 1F3FD 62      chars=6 code points=4 graphemes=3
charAt(1) = d83d (half of a surrogate pair)
```

Thumbs-up + skin tone: 6 chars, 4 code points, 3 graphemes.

Four normalization forms

codeoutput/Forms.java — Java

```
4  public class Forms {
5  static String hex(String s) {                    // code points
       as hex
6      StringBuilder b = new StringBuilder();
7      s.codePoints().forEach(c -> b.append(String.format("%X ", c))
           );
8      return b.toString().trim();
9  }
10
11     public static void main(String[] args) {
12         String[] samples = {"\u00E9", "e\u0301", "\uFB01le", "\
           uFF21\uFF22\uFF11", "x\u00B2", "\u2163"};
13         System.out.printf("%-14s %-14s %-14s %s%n", "input", "NFC
           ", "NFD", "NFKC");
14         for (String s : samples)
15             System.out.printf("%-14s %-14s %-14s %s%n", hex(s),
16                 hex(Normalizer.normalize(s, Form.NFC)), hex(
               Normalizer.normalize(s, Form.NFD)),
17                 hex(Normalizer.normalize(s, Form.NFKC)));
18     }
19 }
```

NFC composes, **NFD** decomposes. The **K** forms also replace *compatibility* characters: ligature `FB01` → `fi`, full-width `FF21 FF22 FF11` → `AB1`, x^2 → `x2`. Store NFC; compare identifiers (user names) after NFKC.

Four normalization forms

codeoutput/Forms.java — Java

```
4  public class Forms {
5  static String hex(String s) {                    // code points
       as hex
6      StringBuilder b = new StringBuilder();
7      s.codePoints().forEach(c -> b.append(String.format("%X ", c))
           );
8      return b.toString().trim();
9  }
10
11     public static void main(String[] args) {
12         String[] samples = {"\u00E9", "e\u0301", "\uFB01le", "\
           uFF21\uFF22\uFF11", "x\u00B2", "\u2163"};
13         System.out.printf("%-14s %-14s %-14s %s%n", "input", "NFC
           ", "NFD", "NFKC");
14         for (String s : samples)
15             System.out.printf("%-14s %-14s %-14s %s%n", hex(s),
16                 hex(Normalizer.normalize(s, Form.NFC)), hex(
           Normalizer.normalize(s, Form.NFD)),
17                 hex(Normalizer.normalize(s, Form.NFKC)));
18     }
19 }
```

Terminal — codeoutput/Forms.txt

```
input          NFC            NFD            NFKC
E9             E9             65 301         E9
65 301         E9             65 301         E9
FB01 6C 65     FB01 6C 65     FB01 6C 65     66 69 6C 65
FF21 FF22 FF11 FF21 FF22 FF11 FF21 FF22 FF11 41 42 31
78 B2          78 B2          78 B2          78 32
2163           2163           2163           49 56
```

NFC composes, **NFD** decomposes. The **K** forms also replace *compatibility* characters: ligature `FB01` → `fi`, full-width `FF21 FF22 FF11` → `AB1`, $\texttt{x}^2$ → `x2`. Store NFC; compare identifiers (user names) after NFKC.

Case mapping is not one-to-one

codeoutput/Mapping.java — Java

```
10      public static void main(String[] args) {
11          Locale tr = Locale.of("tr");
12          String[] words = {"straße", "\uFB01le", "İstanbul", "DİYARBAKIR"};
13          for (String w : words) {
14              String up = w.toUpperCase(Locale.ROOT), low = w.toLowerCase(Locale.ROOT);
15              System.out.printf("%-26s upper=%-11s lower(tr)=%-11s length %d -> %d / %d%n",
16                  hex(w), up, w.toLowerCase(tr), w.length(), up.length(), low.length());
17          }
18          System.out.println("İ lower(ROOT): " + hex("İ".toLowerCase(Locale.ROOT)));
19          System.out.println("char level:    " + Character.toUpperCase('ß') + " (no change)");
20          System.out.println("straße equalsIgnoreCase STRASSE: " + "straße".equalsIgnoreCase("STRASSE"));
21      }
22  }
```

The length can change (ß → SS); İ lower-cased with `ROOT` is `i` + U+0307; `equalsIgnoreCase` works char by char and misses ß/SS.

Case mapping is not one-to-one

codeoutput/Mapping.java — Java

```java
    public static void main(String[] args) {
        Locale tr = Locale.of("tr");
        String[] words = {"straße", "\uFB01le", "İstanbul", "DİYARBAKIR"};
        for (String w : words) {
            String up = w.toUpperCase(Locale.ROOT), low = w.toLowerCase(Locale.ROOT);
            System.out.printf("%-26s upper=%-11s lower(tr)=%-11s length %d -> %d / %d%n",
                hex(w), up, w.toLowerCase(tr), w.length(), up.length(), low.length());
        }
        System.out.println("İ lower(ROOT): " + hex("İ".toLowerCase(Locale.ROOT)));
        System.out.println("char level:    " + Character.toUpperCase('ß') + " (no change)");
        System.out.println("straße equalsIgnoreCase STRASSE: " + "straße".equalsIgnoreCase("STRASSE"));
    }
}
```

Terminal — codeoutput/Mapping.txt

```
73 74 72 61 DF 65          upper=STRASSE     lower(tr)=straße      length 6 -> 7 / 6
FB01 6C 65                 upper=FILE        lower(tr)=file         length 3 -> 4 / 3
130 73 74 61 6E 62 75 6C   upper=İSTANBUL    lower(tr)=istanbul    length 8 -> 8 / 9
44 130 59 41 52 42 41 4B 49 52 upper=DİYARBAKIR  lower(tr)=diyarbakır  length 10 -> 10 / 11
İ lower(ROOT): 69 307
char level:    ß (no change)
straße equalsIgnoreCase STRASSE: false
```

The length can change (ß → SS); İ lower-cased with `ROOT` is i + U+0307; `equalsIgnoreCase` works char by char and misses ß/SS.

Locale Data in Depth

Unicode locale extensions: -u-

codeoutput/Extensions.java — Java

```
8  public class Extensions {
9      public static void main(String[] args) {
10         LocalDate day = LocalDate.of(2026, 10, 15);
11         String[] tags = {"en-US", "en-US-u-ca-buddhist", "en-US-u-ca-japanese",
12                          "tr-TR-u-ca-islamic-umalqura", "en-US-u-cu-eur", "en-US-u-fw-mon",
13                          "en-US-u-rg-trzzzz"};
14         for (String t : tags) {
15             Locale l = Locale.forLanguageTag(t);
16             System.out.printf("%-28s %-28s %-11s %s%n", t,
17                 day.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).localizedBy(l)),
18                 NumberFormat.getCurrencyInstance(l).format(1234.5),
19                 WeekFields.of(l).getFirstDayOfWeek());
20         }
21     }
22 }
```

ca calendar, cu currency, fw first day, rg region; localizedBy (Java 10) applies them.

Unicode locale extensions: -u-

codeoutput/Extensions.java — Java

```java
public class Extensions {
    public static void main(String[] args) {
        LocalDate day = LocalDate.of(2026, 10, 15);
        String[] tags = {"en-US", "en-US-u-ca-buddhist", "en-US-u-ca-japanese",
                         "tr-TR-u-ca-islamic-umalqura", "en-US-u-cu-eur", "en-US-u-fw-mon",
                         "en-US-u-rg-trzzzz"};
        for (String t : tags) {
            Locale l = Locale.forLanguageTag(t);
            System.out.printf("%-28s %-28s %-11s %s%n", t,
                day.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG).localizedBy(l)),
                NumberFormat.getCurrencyInstance(l).format(1234.5),
                WeekFields.of(l).getFirstDayOfWeek());
        }
    }
}
```

Terminal — codeoutput/Extensions.txt

```
en-US                        October 15, 2026             $1,234.50   SUNDAY
en-US-u-ca-buddhist          October 15, 2569 BE          $1,234.50   SUNDAY
en-US-u-ca-japanese          October 15, 8 Reiwa          $1,234.50   SUNDAY
tr-TR-u-ca-islamic-umalqura  Hicri 4 Cemaziyelevvel 1448  ₺1.234,50   MONDAY
en-US-u-cu-eur               October 15, 2026             €1,234.50   SUNDAY
en-US-u-fw-mon               October 15, 2026             $1,234.50   MONDAY
en-US-u-rg-trzzzz            October 15, 2026             TRY1,234.50 MONDAY
```

`ca` calendar, `cu` currency, `fw` first day, `rg` region; `localizedBy` (Java 10) applies them.

Two default locales

codeoutput/Categories/Cat.java **Java**

```java
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;
import java.util.Locale.Category;

public class Cat {
    public static void main(String[] args) {
        System.out.println("DISPLAY=" + Locale.getDefault(
        Category.DISPLAY)
            + " FORMAT=" + Locale.getDefault(Category.FORMAT));
        System.out.println(Locale.FRANCE.getDisplayName()
                // uses DISPLAY
            + " | " + String.format("%,.2f", 1234.5)
                // uses FORMAT
            + " | " + LocalDate.of(2026, 10, 15).format(
                DateTimeFormatter.ofLocalizedDate(FormatStyle.
        LONG)));  // uses FORMAT
    }
}
```

`Locale.Category.DISPLAY` names things (languages, months in menus), `FORMAT` formats values. A user may want English menus with Turkish numbers: `-Duser.language.format=tr` (or `Locale.setDefault(Category.FORMAT, l)`).

Two default locales

codeoutput/Categories/Cat.java — Java

```java
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;
import java.util.Locale.Category;

public class Cat {
    public static void main(String[] args) {
        System.out.println("DISPLAY=" + Locale.getDefault(
        Category.DISPLAY)
            + " FORMAT=" + Locale.getDefault(Category.FORMAT));
        System.out.println(Locale.FRANCE.getDisplayName()
                // uses DISPLAY
            + " | " + String.format("%,.2f", 1234.5)
                // uses FORMAT
            + " | " + LocalDate.of(2026, 10, 15).format(
                DateTimeFormatter.ofLocalizedDate(FormatStyle.
        LONG)));  // uses FORMAT
    }
}
```

Terminal — codeoutput/Categories.txt

```
$ java Cat.java
DISPLAY=en_US FORMAT=en_US
French (France) | 1,234.50 | October 15, 2026
$ java -Duser.language.format=tr -Duser.country.format=TR Cat.java
DISPLAY=en_US FORMAT=tr_TR
French (France) | 1.234,50 | 15 Ekim 2026
$ java -Duser.language.display=de Cat.java
DISPLAY=de_US FORMAT=en_US
Französisch (Frankreich) | 1,234.50 | October 15, 2026
```

`Locale.Category.DISPLAY` names things (languages, months in menus), `FORMAT` formats values. A user may want English menus with Turkish numbers: `-Duser.language.format=tr` (or `Locale.setDefault(Category.FORMAT, l)`).

Locale data providers: CLDR and COMPAT

codeoutput/Providers/Prov.java **Java**

```java
import java.text.NumberFormat;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;

public class Prov {
    public static void main(String[] args) {
        Locale ch = Locale.forLanguageTag("de-CH");
        System.out.println(NumberFormat.getInstance(ch).format
        (1234567.5) + " | "
            + NumberFormat.getCompactNumberInstance(Locale.
        GERMAN, NumberFormat.Style.LONG)
                .format(2_000_000) + " | "
            + LocalDate.of(2026, 10, 15).format(
        DateTimeFormatter
                .ofLocalizedDate(FormatStyle.FULL).withLocale(
        Locale.of("tr"))));
    }
}
```

Since Java 9 the data comes from **CLDR**. The old JRE data (`COMPAT`) is deprecated in Java 21 and removed in Java 23. Upgrading Java can change formatted output: never parse your own UI text in tests.

Locale data providers: CLDR and COMPAT

codeoutput/Providers/Prov.java — Java

```java
import java.text.NumberFormat;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.FormatStyle;
import java.util.Locale;

public class Prov {
    public static void main(String[] args) {
        Locale ch = Locale.forLanguageTag("de-CH");
        System.out.println(NumberFormat.getInstance(ch).format
        (1234567.5) + " | "
            + NumberFormat.getCompactNumberInstance(Locale.
        GERMAN, NumberFormat.Style.LONG)
                .format(2_000_000) + " | "
            + LocalDate.of(2026, 10, 15).format(
        DateTimeFormatter
                .ofLocalizedDate(FormatStyle.FULL).withLocale(
        Locale.of("tr"))));
    }
}
```

Terminal — codeoutput/Providers.txt

```
$ java -XshowSettings:locale -version 2>&1 | head -4
Locale settings:
    default locale = English (United States)
    default display locale = English (United States)
    default format locale = English (United States)
$ java Prov.java
1'234'567.5 | 2 Millionen | 15 Ekim 2026 Perşembe
$ java -Djava.locale.providers=COMPAT Prov.java
Oct 15, 2026 12:11:10 PM sun.util.locale.provider.
        LocaleProviderAdapter <clinit>
WARNING: COMPAT locale provider will be removed in a future release
1'234'567.5 | 2 million | 15 Ekim 2026 Perşembe
```

Since Java 9 the data comes from **CLDR**. The old JRE data (`COMPAT`) is deprecated in Java 21 and removed in Java 23. Upgrading Java can change formatted output: never parse your own UI text in tests.

Plural Rules

CLDR plural rules in the JDK

codeoutput/Plurals.java — Java

```
4  public class Plurals {
5      public static void main(String[] args) {
6          long[] counts = {1, 2, 5, 22, 25};
7          for (String tag : new String[] {"en", "tr", "de", "fr", "pl"}) {
8              NumberFormat f = NumberFormat.getCompactNumberInstance(
9                  Locale.forLanguageTag(tag), NumberFormat.Style.LONG);
10             StringBuilder line = new StringBuilder(String.format("%-3s", tag));
11             for (long c : counts) {
12                 String s = f.format(c * 1_000_000);
13                 line.append(String.format(" | %-17s", s));
14             }
15             System.out.println(line);
16         }
17     }
18 }
```

English and German: 2 forms; Turkish uses one; Polish has 3 (*milion*, *miliony*, *milionów*: 1 / 2–4, 22 / 5, 25). `ChoiceFormat` cannot express Polish or Arabic rules; use ICU4J or keep messages simple.

CLDR plural rules in the JDK

codeoutput/Plurals.java — Java

```java
public class Plurals {
    public static void main(String[] args) {
        long[] counts = {1, 2, 5, 22, 25};
        for (String tag : new String[] {"en", "tr", "de", "fr", "pl"}) {
            NumberFormat f = NumberFormat.getCompactNumberInstance(
                Locale.forLanguageTag(tag), NumberFormat.Style.LONG);
            StringBuilder line = new StringBuilder(String.format("%-3s", tag));
            for (long c : counts) {
                String s = f.format(c * 1_000_000);
                line.append(String.format(" | %-17s", s));
            }
            System.out.println(line);
        }
    }
}
```

Terminal — codeoutput/Plurals.txt

```
en  | 1 million         | 2 million         | 5 million         | 22 million        | 25 million
tr  | 1 milyon          | 2 milyon          | 5 milyon          | 22 milyon         | 25 milyon
de  | 1 Million         | 2 Millionen       | 5 Millionen       | 22 Millionen      | 25 Millionen
fr  | 1 million         | 2 millions        | 5 millions        | 22 million        | 25 million
pl  | 1 milion          | 2 miliony         | 5 milionów        | 22 milionów       | 25 milionów
```

English and German: 2 forms; Turkish uses one; Polish has 3 (*milion*, *miliony*, *milionów*: 1 / 2–4, 22 / 5, 25). `ChoiceFormat` cannot express Polish or Arabic rules; use ICU4J or keep messages simple.

Messages with real plural rules

```
// ICU4J (com.ibm.icu:icu4j), not in the JDK
MessageFormat m = new MessageFormat(
    "{n, plural, one {# dosya} other {# dosya}}",
    ULocale.forLanguageTag("tr"));
// Polish: one / few / many / other
"{n, plural, one {# plik} few {# pliki} many {# plików}
     other {# pliku}}"
```

Not run here (ICU4J comes from Maven Central).

- CLDR categories: `zero`, `one`, `two`, `few`, `many`, `other`.
- Each language uses some of them; the translator fills in the forms.
- The JDK has the rules (used by `CompactNumberFormat`) but no public plural API; `java.text.ListFormat` (*a, b and c*) arrived in Java 22.

Bundle Lookup in Detail

Candidate locales

codeoutput/Candidates.java — Java

```java
import java.util.Locale;
import java.util.ResourceBundle.Control;

public class Candidates {
    public static void main(String[] args) {
        Control c = Control.getControl(Control.FORMAT_DEFAULT);
        for (String tag : new String[] {"tr-TR", "de-CH-1996", "sr-Latn-RS", "zh-TW", "nb-NO"})
            System.out.printf("%-11s %s%n", tag,
                c.getCandidateLocales("Messages", Locale.forLanguageTag(tag)));
        System.out.println("bundle name: "
            + c.toBundleName("Messages", Locale.forLanguageTag("sr-Latn-RS")));
    }
}
```

Script and variant are tried first; special cases: `zh-TW` implies the script `Hant`, Norwegian Bokmål `nb` also tries the old code `no`. The empty locale is the base bundle.

Candidate locales

codeoutput/Candidates.java — Java

```java
import java.util.Locale;
import java.util.ResourceBundle.Control;

public class Candidates {
    public static void main(String[] args) {
        Control c = Control.getControl(Control.FORMAT_DEFAULT);
        for (String tag : new String[] {"tr-TR", "de-CH-1996", "sr-Latn-RS", "zh-TW", "nb-NO"})
            System.out.printf("%-11s %s%n", tag,
                c.getCandidateLocales("Messages", Locale.forLanguageTag(tag)));
        System.out.println("bundle name: "
            + c.toBundleName("Messages", Locale.forLanguageTag("sr-Latn-RS")));
    }
}
```

Terminal — codeoutput/Candidates.txt

```
tr-TR       [tr_TR, tr, ]
de-CH-1996  [de_CH_1996, de_CH, de, ]
sr-Latn-RS  [sr_RS_#Latn, sr__#Latn, sr_RS, sr, ]
zh-TW       [zh_TW_#Hant, zh__#Hant, zh_TW, zh, ]
nb-NO       [nb_NO, no_NO, nb, no, ]
bundle name: Messages_sr_Latn_RS
```

Script and variant are tried first; special cases: `zh-TW` implies the script `Hant`, Norwegian Bokmål `nb` also tries the old code `no`. The empty locale is the base bundle.

ResourceBundle.getBundle

Algorithm 1: GetBundle(b, ℓ) (default `Control`)

Input: base name b, locale ℓ, default locale δ, class loader

1 **for** $t \leftarrow \ell, \delta$ **do**
2 **foreach** *c in* CandidateLocales(b, t) **do**
  `// e.g. tr_TR, tr, the empty locale`
3 **if** *the cache has bundle* (b, c) **then** use it
4 **else** try class `b_c`, then file `b_c.properties`; cache the result (also "not found")
5 link the found bundles: each one's parent is the next found candidate
6 **if** *a bundle other than the base was found for* t **then return** *the first one*
7 **if** $t = \delta$ *or* FallbackLocale$(b, \ell) =$ *none* **then break**
8 **return** *the base bundle, or throw* `MissingResourceException`

Bundles are **cached**; `ResourceBundle.clearCache()` or a `Control` with a time-to-live reloads them.

Time Zones in Depth

Time zone rules change

codeoutput/TimeRules.java — Java

```
 8 public class TimeRules {
 9     public static void main(String[] args) {
10         System.out.println("time zone data: " + ZoneRulesProvider.getVersions("UTC").lastKey());
11         ZoneRules ist = ZoneId.of("Europe/Istanbul").getRules();
12         for (String t : new String[] {"2015-01-15", "2015-07-15", "2017-01-15", "2026-07-15"})
13             System.out.println("Istanbul " + t + ": UTC" + ist.getOffset(Instant.parse(t + "T12:00:00Z")));
14
15         LocalDateTime twice = LocalDateTime.of(2026, 10, 25, 2, 30);    // clocks go back at 03:00
16         ZoneId berlin = ZoneId.of("Europe/Berlin");
17         System.out.println("Berlin 02:30 offsets: " + berlin.getRules().getValidOffsets(twice));
18         ZonedDateTime first = ZonedDateTime.of(twice, berlin);
19         System.out.println("earlier: " + first.toInstant() + "  later: "
20             + first.withLaterOffsetAtOverlap().toInstant());
21     }
22 }
```

Zone rules are law (Türkiye: no clock changes since 2016). Autumn: 02:30 happens twice.

Time zone rules change

codeoutput/TimeRules.java — Java

```java
public class TimeRules {
    public static void main(String[] args) {
        System.out.println("time zone data: " + ZoneRulesProvider.getVersions("UTC").lastKey());
        ZoneRules ist = ZoneId.of("Europe/Istanbul").getRules();
        for (String t : new String[] {"2015-01-15", "2015-07-15", "2017-01-15", "2026-07-15"})
            System.out.println("Istanbul " + t + ": UTC" + ist.getOffset(Instant.parse(t + "T12:00:00Z")));

        LocalDateTime twice = LocalDateTime.of(2026, 10, 25, 2, 30);    // clocks go back at 03:00
        ZoneId berlin = ZoneId.of("Europe/Berlin");
        System.out.println("Berlin 02:30 offsets: " + berlin.getRules().getValidOffsets(twice));
        ZonedDateTime first = ZonedDateTime.of(twice, berlin);
        System.out.println("earlier: " + first.toInstant() + "  later: "
            + first.withLaterOffsetAtOverlap().toInstant());
    }
}
```

Terminal — codeoutput/TimeRules.txt

```
time zone data: 2026b
Istanbul 2015-01-15: UTC+02:00
Istanbul 2015-07-15: UTC+03:00
Istanbul 2017-01-15: UTC+03:00
Istanbul 2026-07-15: UTC+03:00
Berlin 02:30 offsets: [+02:00, +01:00]
earlier: 2026-10-25T00:30:00Z  later: 2026-10-25T01:30:00Z
```

Zone rules are law (Türkiye: no clock changes since 2016). Autumn: 02:30 happens twice.

Which type for what?

type	meaning	use for
`Instant`	a point on the time line (UTC)	logs, database, events
`ZonedDateTime`	instant + zone rules	meetings, display, arithmetic over DST
`OffsetDateTime`	instant + fixed offset	data exchange (ISO 8601)
`LocalDateTime`	wall clock, no zone	"every day at 09:00", forms
`LocalDate`	a day	birthdays, holidays

- Store `Instants`; keep the user's `ZoneId` as a preference; convert only to show.
- For a future appointment store the local time *and* the zone: the law may change before the date.

Formatters and Threads

Formatters and threads

codeoutput/Unsafe.java — Java

```java
public class Unsafe {
    public static void main(String[] args) throws Exception {
        SimpleDateFormat old = new SimpleDateFormat("yyyy-MM-dd");      // NOT thread-safe
        DateTimeFormatter modern = DateTimeFormatter.ofPattern("yyyy-MM-dd"); // immutable
        AtomicInteger badOld = new AtomicInteger(), badNew = new AtomicInteger();
        IntStream.range(0, 20_000).parallel().forEach(i -> {
            LocalDate d = LocalDate.of(2000 + i % 30, 1 + i % 12, 1 + i % 28);
            String expected = d.toString();
            try {
                Date parsed = old.parse(expected);
                if (!old.format(parsed).equals(expected)) badOld.incrementAndGet();
            } catch (Exception e) { badOld.incrementAndGet(); }
            if (!LocalDate.parse(expected, modern).format(modern).equals(expected))
                badNew.incrementAndGet();
        });
        System.out.println("wrong with SimpleDateFormat:  " + (badOld.get() > 0 ? "yes" : "no"));
        System.out.println("wrong with DateTimeFormatter: " + (badNew.get() > 0 ? "yes" : "no"));
    }
}
```

`SimpleDateFormat` keeps state: shared between threads it mixes results. `DateTimeFormatter` is immutable.

Formatters and threads

codeoutput/Unsafe.java — Java

```java
public class Unsafe {
    public static void main(String[] args) throws Exception {
        SimpleDateFormat old = new SimpleDateFormat("yyyy-MM-dd");      // NOT thread-safe
        DateTimeFormatter modern = DateTimeFormatter.ofPattern("yyyy-MM-dd"); // immutable
        AtomicInteger badOld = new AtomicInteger(), badNew = new AtomicInteger();
        IntStream.range(0, 20_000).parallel().forEach(i -> {
            LocalDate d = LocalDate.of(2000 + i % 30, 1 + i % 12, 1 + i % 28);
            String expected = d.toString();
            try {
                Date parsed = old.parse(expected);
                if (!old.format(parsed).equals(expected)) badOld.incrementAndGet();
            } catch (Exception e) { badOld.incrementAndGet(); }
            if (!LocalDate.parse(expected, modern).format(modern).equals(expected))
                badNew.incrementAndGet();
        });
        System.out.println("wrong with SimpleDateFormat:  " + (badOld.get() > 0 ? "yes" : "no"));
        System.out.println("wrong with DateTimeFormatter: " + (badNew.get() > 0 ? "yes" : "no"));
    }
}
```

Terminal — codeoutput/Unsafe.txt

```
wrong with SimpleDateFormat:  yes
wrong with DateTimeFormatter: no
```

`SimpleDateFormat` keeps state: shared between threads it mixes results. `DateTimeFormatter` is immutable.

Rules for formatters

class	thread-safe?	how to share
`DateTimeFormatter`	yes	`static final`
`Locale`, `Currency`, `ZoneId`	yes	anywhere
`ResourceBundle`	yes (read only)	cache per locale
`NumberFormat`, `DecimalFormat`	no	new per use, or `ThreadLocal`
`SimpleDateFormat`, `MessageFormat`	no	replace by `java.time`, new per use
`Collator`	no	`clone()` per thread; `CollationKey` for big sorts

`getInstance(l)` methods return a *new* object each time, built from cached data: cheap enough for most code.

Text Direction

Right-to-left languages

- Arabic, Hebrew, Persian, Urdu are written **right to left**; numbers and Latin words inside them still run left to right: *bidirectional* text.
- The **Unicode Bidi Algorithm** decides the display order; the stored order is the reading order.
- `java.text.Bidi` computes the levels; Swing uses `ComponentOrientation.getOrientation(locale)` to mirror whole windows.

Design for it

- use *start*/*end*, not *left*/*right*
- mirror arrows and progress bars
- never build a sentence from fragments: the order changes
- test with a right-to-left locale (`ar`, `he`)

Summary

Big ideas

- A string has bytes, `chars`, code points and graphemes: count what you mean.
- Normalize (NFC to store, NFKC to compare identifiers) before comparing text from outside.
- Case mapping may change the length and depends on the language.
- Locales carry extensions (calendar, currency, first day, numbering); `localizedBy` uses them.
- Locale data comes from CLDR and changes between Java versions; time zone rules change by law.
- Plural rules differ widely; the JDK has them only inside `CompactNumberFormat`.
- Know which formatters are thread-safe; prefer `java.time`.

Exercises

1. Write `truncate(s, n)` that keeps at most `n` graphemes and never cuts a surrogate pair.
2. Check whether two user names are "the same" (NFKC + case folding). Which pairs surprise you?
3. Print next month's calendar with the right first day of the week for `en-US`, `tr-TR`, `ar-SA`.
4. Find a date and zone where `LocalDateTime` + one day $\neq$ `ZonedDateTime` + one day.
5. Make `Unsafe` correct with a `ThreadLocal<SimpleDateFormat>`; measure the cost.
6. Run your sophomore project with `-Djava.locale.providers=COMPAT`: what changes?

Questions?